

Welche Definitionen zur Aussagenlogik sind korrekt?

Wählen Sie eine oder mehrere Antworten:

- ☐ a. Die Ausdrücke $C \vee A \wedge B$, $((A) \vee B)$ und $A B$ sind syntaktisch wohlgeformt.
- ☒ b. Die Belegung bzw. Interpretation einer Aussagenlogik meint, dass man jeder Aussagenvariable einen Wahrheitswert (wahr oder falsch) zuweist.
- ☒ c. $A \Rightarrow B$ ist nur falsch, wenn die erste Aussage A (Vordersatz, Implikans) wahr und die zweite Aussage (Hintersatz, Implikat) falsch ist. Die Implikation kann auch über die Negation und die Disjunktion $(\neg A \vee B)$ ausgedrückt werden.
- ☒ d. Die Syntax einer Aussagenlogik bestimmt die Wohlgeformtheit von Ausdrücken, die aus Symbolen und logischen Operatoren bestehen sowie beginnend mit atomaren Formeln rekursiv definiert werden.
- ☐ e. Eine aussagenlogische Formel mit n Variablen besitzt $2 * n$ verschiedene Belegungen.
- ☐ f. Bei komplexeren Formeln ist es notwendig, eine Reihenfolge der Operatoren festzulegen. Dabei hat die Negation Vorrang vor der Klammerung, dieser wiederum vor der Konjunktion, der Disjunktion, der Äquivalenz und schließlich der Implikation.
- ☐ g. $A \wedge B$ stellt eine atomare, aussagenlogische Formel dar.

Die Schließen aus einer Wissensbasis auf neue Aussagen nennt man ...

Folgerung

Können mehrere aussagenlogische Formeln nicht gleichzeitig erfüllt werden, so nennt man dies ...

Widerspruch

Eine ausdrucksstärkere Logik als die Aussagenlogik heißt ...

Prädikatenlogik

Die Abbildung von Expertenwissen in Form von aussagenlogischer Formeln nennt man ...

Wissensrepräsentation

Eine digitale Schaltung ist verifiziert man aufgrund des großen Umfangs in der Regel durch ...

Automatisierte Beweiser

Input:

Assume that the input is a set of facts specified by the following predicates:

- adjacent(X, Y): country X is adjacent to country Y
- eu(X): country X is a member of the European Union
- candidate(X): country X is a candidate member
- euro(X): country X uses the Euro
- schengen(X): country X belongs to the Schengen area

Note: The name of a state is given by a string constant.

Task:

Write an ASP program that computes all Schengen countries that share a border with a Non-Schengen country. For each such country C, derive an atom outBorder(C).

zum Beispiel:

Test	Result
show outBorder(1).	outBorder("Bulgaria") outBorder("Lithuania") outBorder("Finland") outBorder("France") outBorder("Greece") outBorder("Hungary") outBorder("Italy") outBorder("Latvia") outBorder("Lithuania") outBorder("Poland") outBorder("Slovakia") outBorder("Slovenia") outBorder("Spain")

Antwort: (Abgabesystem: 0 %)

Antwort zurücksetzen

```
1 show outBorder(1).
2 outBorder(C) :- schengen(C), adjacent(C, Y), not schengen(Y).
3
4
5 show outBorder(1).
```

Input:

The input defines the connections between two stations in the public transport system in Vienna. It is a set of facts of form `conn(From, To, Type, Line)`.

where `From` and `To` are quoted station names, `Type` declares the type of transportation (bus, tram, train or metro; all without quotes), and `Line` specifies the quoted name of the line.

Examples:

- `conn("Vorgartenstraße","Donauinsel",metro,"U1").`
- `conn("Siedlung Sonnheim","Eisstoßgasse",bus,"93A").`
- `conn("Aumannplatz","Weinhauser Gasse",tram,"41").`
- `conn("Laa an der Thaya Bahnhof","Kottingneusiedl Bahnhof",train,"S2").`

Moreover, assume that a fact of form `start(S)`, where `S` is a station is, is given.

Task:

Write an ASP program that computes all nodes reachable from `S` by metro with arbitrary many line changes but at most three stops (excluding `S` but including the destination). The result is to be encoded using atoms of form `reachable(.)`.

You can (but do not have to) use helper predicates, but make sure to keep the `#show` directives in the answer box!

Zum Beispiel:

Test	Resultat
<code>start("Floridsdorf").</code>	<code>reachable("Dresdner-Straße") reachable("Floridsdorf") reachable("Handelskai") reachable("Neue-Donau")</code>

Antwort: (Abzugssystem: 0 %)

Antwort zurücksetzen

1	<code>reachable(To, 1) :- start(S), conn(S, To, metro, _).</code>
2	
3	<code>reachable(To, N+1) :- reachable(From, N), N < 3, conn(From, To, metro, _), not To = From.</code>
4	
5	<code>reachable(Station) :- reachable(Station, N), N <= 3.</code>
6	<code>#show reachable/1.</code>

Input:

Assume that the input is a set of facts specified by the following predicates:

- `childOf(X, Y)`: `X` is a child of `Y`
- `female(X)`: `X` is female
- `male(X)`: `X` is male

Task:

Write an ASP program that derives atoms over the following predicates:

- `father(X, Y)`: whenever `X` is the father of `Y`
- `mother(X, Y)`: whenever `X` is the mother of `Y`

You can (but do not have to) use helper predicates, but make sure to keep the `#show` directives in the answer box!

Zum Beispiel:

Test	Resultat
<code>childOf(bart,homer). childOf(lisa,homer). childOf(homer,abe). female(lisa). male(bart). male(homer). male(abe).</code>	<code>father(abe,homer) father(homer,bart) father(homer,lisa)</code>

Antwort: (Abzugssystem: 0 %)

Antwort zurücksetzen

1	<code>father(X, Y) :- male(X), childOf(Y, X).</code>
2	
3	<code>mother(X, Y) :- female(X), childOf(Y, X).</code>
4	
5	<code>#show father/2.</code>
6	<code>#show mother/2.</code>